



CS1004 – Object Oriented Programming

Lecture # 10
Tuesday, March 08, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Introduction to OOP

- Object-oriented programming (OOP)
 - Encapsulates data (attributes) and functions (behavior) into packages called classes
- Classes
 - Classes are the standard unit of programming
 - A class is like a blueprint – reusable
 - Objects are instantiated (created) from the class

Basic Syntax

➤ In C++:

➤ `class` `class_name`{ `Member_list` }; //Class

➤ `Member_List` includes

➤ `MemberVariables` and

➤ `MemberFunctions`

➤ `class_name` `identifier`; //Object

Time Class

➤ Classes

- Model objects that have attributes (data members) and behaviors (member functions)
- Defined using keyword class
- Have a body delineated with braces ({ and })
- Class definitions terminate with a semicolon

```
class Time{  
public:  
    Time();  
    void setTime(int, int, int);  
    void printMilitary();  
    void printStandard();  
private:  
    int hour = 0; // 0 - 23  
    int minute = 0; // 0 - 59  
    int second = 0; // 0 - 59  
};
```

Public: and **Private:** are member-access specifiers.

setTime, **printMilitary**, and **printStandard** are **member functions**.
Time is the **constructor**.

hour, **minute**, and **second** are **data members**.

Member Access Specifiers

➤ public

- Presents clients with a view of the services the class provides (interface)
- Data and member functions are accessible

➤ private

- Default access mode
- Data only accessible to member functions and friends
- private members only accessible through the public class interface using public member functions

Member Functions

➤ Inline

- The functions are defined within the body of the class definition

➤ Out-of-line:

- The functions are declared within the body of the class definition and defined outside

Member Functions

- If a member function is defined outside the class
 - Scope resolution operator (::) and class name are needed
 - Defining a function outside a class does not change it being **public** or **private**
- Binary scope resolution operator (::)
 - Combines the class name with the member function name
 - Different classes can have member functions with the same name
- Format for defining member functions

```
returnType ClassName::MemberFunctionName( ){  
    ...  
}
```

Time Class Function definition

```
#include <iostream>

using namespace std;

// Time abstract data type (ADT) definition
class Time{
public:
    Time();//constructor
    void setTime(int, int, int);//set hour, minute, second
    void printMilitary();//print military time
    void printStandard();//print standard time
private:
    int hour = 0;//0 - 23
    int minute = 0;//0 - 59
    int second = 0;//0 - 59
};

//Time constructor initializes each data member to zero
//Ensures all Time objects start in a consistent state
Time::Time() { hour = minute = second = 0; }
```

```
//Set a new Time value using military time. Perform validity
//checks on the data values. Set invalid values to zero.
void Time::setTime(int h, int m, int s) {
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

//Print Time in military format
void Time::printMilitary(){
    cout << (hour < 10 ? "0" : "") << hour << ":"
         << (minute < 10 ? "0" : "") << minute;
}

//Print Time in standard format
void Time::printStandard(){
    cout << ((hour == 0 || hour == 12) ? 12 : hour % 12)
         << ":" << (minute < 10 ? "0" : "") << minute
         << ":" << (second < 10 ? "0" : "") << second
         << (hour < 12 ? " AM" : " PM");
}
```

Creating objects of the Time class

➤ Class definition and declaration

- Once a class has been defined, it can be used as a type in object, array and pointer declarations

➤ Example:

```
Time  sunset, // object of type Time
      arrayOfTimes[ 5 ], // array of Time objects
      *pointerToTime, // pointer to a Time object
      &dinnerTime = sunset; // reference to a Time object
```

Note: The class name becomes the new type specifier.

Time Class: Driver program

```
// Driver to test simple class Time
int main()
{
    Time t; // instantiate object t of class Time

    cout << "The initial military time is ";
    t.printMilitary();

    cout << "The initial standrd time is ";
    t.printStandard();
```

The initial military time is 00:00
The initial standard time is 12:00:00 AM

Notice how functions
are called using the dot
(.) operator.

```

t.setTime(13, 27, 15);
cout << "\n\nMilitary time after setTime is ";
t.printMilitary();
cout << "\n\nStandard time after setTime is ";
t.printStandard();

```

```

Military time after setTime is 13:27
Standard time after setTime is 1:27:06 PM

```

```

t.setTime(99, 99, 99); // attempt invalid settings
cout << "\n\nAfter attempting invalid settings:"
    << "\nMilitary time: ";
t.printMilitary();
cout << "\nStandard time: ";
t.printStandard();
cout << endl;
return 0;
}

```

```

After attempting invalid settings:
Military time: 00:00
Standard time: 12:00:00 AM

```

```

The initial military time is 00:00
The initial standard time is 12:00:00 AM

Military time after setTime is 13:27
Standard time after setTime is 1:27:06 PM

After attempting invalid settings:
Military time: 00:00
Standard time: 12:00:00 AM

```

Initializing Class Objects: Constructors

```
//Time constructor initializes each data member to zero  
//Ensures all Time objects start in a consistent state  
Time::Time() { hour = minute = second = 0; }
```

Initializing Class Objects: Constructors

➤ Constructors

- Initialize class members
- Same name as the class
- No return type
- Member variables can be initialized by the constructor or set afterwards

➤ Passing arguments to a constructor

- When an object of a class is declared, initializers can be provided
- Format of declaration with initializers:
`Class-type ObjectName( value1,value2,...);`
- Default arguments may also be specified in the constructor prototype

Initializing Class Objects: Constructors

```
// Time abstract data type (ADT) definition
class Time{
public:
    Time(int=0,int=0,int=0); //Default constructor
    void setTime(int, int, int); //set hour, minute, second
    void printMilitary(); //print military time
    void printStandard(); //print standard time
private:
    int hour = 0; //0 - 23
    int minute = 0; //0 - 59
    int second = 0; //0 - 59
};
```

Notice that default settings for the three member variables are set in constructor prototype. No names are needed; the defaults are applied in the order the member variables are declared.

```
// Driver to test simple class Time
int main()
{
    Time t1,           // All arguments default
        t2(2),         // minutes and seconds default
        t3(21, 34),    // seconds default
        t4(12, 25, 42), // all values specified;
        t5(27, 74, 99); // all bad values specified
    cout << "Constructed with:\n"
         << "all arguments defaulted:\n  ";
    t1.printMilitary();
    cout << "\n  ";
    t1.printStandard();

    cout << "\nhour specified; minute and second defaulted:\n";
    t2.printMilitary();
    cout << "\n";
    t2.printStandard();

    cout << "\nhour and minute specified; second defaulted:\n";
    t3.printMilitary();
    cout << "\n";
    t3.printStandard();
}
```

Notice how objects are initialized:
Constructor ObjectName (value1,value2...) ;
If not enough values are specified, the rightmost values are set to their defaults.

```

cout << "\nhour,minute and second specified:\n";
t4.printMilitary();
cout << "\n";
t4.printStandard();

cout << "\nall invalid values specified";

t5.printMilitary();
cout << "\n";
t5.printStandard();
cout << endl;

return 0;
}

```

When only **hour** is specified, **minute** and **second** are set to their default values of 0.

Output

```

Constructed with:
all arguments defaulted:
00:00
12:00:00 AM
hour specified; minute and second defaulted:
02:00
2:00:00 AM
hour and minute specified; second defaulted:
21:34
9:34:00 PM
hour, minute, and second specified:
12:25
12:25:42 PM
all invalid values specified:
00:00
12:00:00 AM

```

Using Destructors

➤ Destructors

- Are member function of class
- Perform termination housekeeping before the system reclaims the object's memory
- Complement of the constructor

- Name is tilde (~) followed by the class name (i.e., ~Time)
 - Recall that the constructor's name is the class name
- Receives no parameters, returns no value
- One destructor per class
 - No overloading allowed

When Constructors and Destructors Are Called

- Constructors and destructors called automatically
 - Order depends on scope of objects
- Global scope objects
 - Constructors called before any other function (including main)
 - Destructors called when main terminates (or **exit** function called)
 - Destructors not called if program terminates with **abort**
- Automatic local objects
 - Constructors called when objects are defined
 - Destructors called when objects leave scope
 - i.e., when the block in which they are defined is exited
 - Destructors not called if the program ends with **exit** or **abort**

Class CreateAndDestroy

```
class CreateAndDestroy {  
public:  
    CreateAndDestroy(int); //Constructor  
    ~CreateAndDestroy(); //Destructor  
private:  
    int data;  
};
```

Constructor and Destructor

```
CreateAndDestroy::CreateAndDestroy(int value)
{
    data = value;
    cout << "Object " << data << " constructor";
}
```

```
CreateAndDestroy::~~CreateAndDestroy()
{
    cout << "Object " << data << " destructor " << endl;
}
```

```

void create(void); // prototype
CreateAndDestroy first(1); // global object

int main()
{
    cout << "    (global created before main)" << endl;
    CreateAndDestroy second(2); // local object
    cout << "    (local automatic in main)" << endl;

    static CreateAndDestroy third(3); // local object
    cout << "    (local static in main)" << endl;

    create(); // call function to create objects

    CreateAndDestroy fourth(4); // local object
    cout << "    (local automatic in main)" << endl;

    return 0;
}

```

```

//Function to create object
void create(void)
{
    CreateAndDestroy fifth(5);
    cout << "    (local automatic in create)"
         << endl;

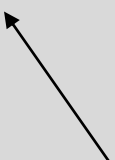
    static CreateAndDestroy sixth(6);
    cout << "    (local static in create)"
         << endl;

    CreateAndDestroy seventh(7);
    cout << "    (local automatic in create)"
         << endl;
}

```

OUTPUT

Object 1	constructor	(global created before main)
Object 2	constructor	(local automatic in main)
Object 3	constructor	(local static in main)
Object 5	constructor	(local automatic in create)
Object 6	constructor	(local static in create)
Object 7	constructor	(local automatic in create)
Object 7	destructor	
Object 5	destructor	
Object 4	constructor	(local automatic in main)
Object 4	destructor	
Object 2	destructor	
Object 6	destructor	
Object 3	destructor	
Object 1	destructor	



Notice how the order of the constructor and destructor call depends on the types of variables (automatic, global and **static**) they are associated with.

Separating interface from implementation

```

1 // Fig. 6.5: time1.h
2 // Declaration of the Time class.
3 // Member functions are defined in time1.cpp
4
5 // prevent multiple inclusions of header file
6 #ifndef TIME1_H
7 #define TIME1_H
8
9 // Time abstract data type definition
10 class Time {
11 public:
12     Time(); // constructor
13     void setTime( int, int, int ); // set hour, minute, second
14     void printMilitary(); // print military time format
15     void printStandard(); // print standard time format
16 private:
17     int hour; // 0 - 23
18     int minute; // 0 - 59
19     int second; // 0 - 59
20 };
21
22 #endif

```

Dot (.) replaced with underscore (_) in file name.

If `time1.h` (`TIME1_H`) is not defined (`#ifndef`) then it is loaded (`#define TIME1_H`). If `TIME1_H` is already defined, then everything up to `#endif` is ignored. This prevents loading a header file multiple times.

```

23 // Fig. 6.5: time1.cpp
24 // Member function definitions for Time class.
25 #include <iostream>
26
27 using std::cout;
28
29 #include "time1.h"
30
31 // Time constructor initializes each data member to zero.
32 // Ensures all Time objects start in a consistent state.
33 Time::Time() { hour = minute = second = 0; }
34
35 // Set a new Time value using military time. Perform validity
36 // checks on the data values. Set invalid values to zero.
37 void Time::setTime( int h, int m, int s )
38 {
39     hour    = ( h >= 0 && h < 24 ) ? h : 0;
40     minute  = ( m >= 0 && m < 60 ) ? m : 0;
41     second  = ( s >= 0 && s < 60 ) ? s : 0;
42 }
43
44 // Print Time in military format
45 void Time::printMilitary()
46 {
47     cout << ( hour < 10 ? "0" : "" ) << hour << ":"
48           << ( minute < 10 ? "0" : "" ) << minute;
49 }
50
51 // Print time in standard format
52 void Time::printStandard()
53 {
54     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
55           << ":" << ( minute < 10 ? "0" : "" ) << minute
56           << ":" << ( second < 10 ? "0" : "" ) << second
57           << ( hour < 12 ? " AM" : " PM" );
58 }

```

Source file uses `#include` to load the header file

Source file contains function definitions

Questions



PresenterMedia